End-of-Studies Project (PFE)

A Privacy-First, No-Code AI Platform
for Tabular ML, Local Generative AI, and Deep Learning

[Student Name]
Supervised by [Supervisor Name]

May 21, 2026

Abstract

This report describes a no-code AI platform built over six sprints as an end-of-studies project. The platform lets non-technical users upload their data, sketch a pipeline on a visual canvas, and train a model under one firm constraint: nothing ever leaves the machine. Three workloads share the same canvas. The first is classical tabular ML across fourteen algorithms. The second is local Generative AI through a Retrieval-Augmented Generation chat powered by Ollama and `pgvector`. The third is deep-learning image classification on a small CNN catalogue sized for a 6 GB consumer GPU. The chapters that follow walk through each sprint's design choices, the things that broke on the first try, and the conventions I leaned on throughout: Celery for anything CPU-bound, MongoDB for anything that resists a schema, and a strict gateway-only header-injection scheme that gave the microservices a zero-trust boundary without the usual gymnastics.

Contents

List of Figures

General Introduction

Machine learning is now routine in almost every industry, but the tooling around it still defaults to one of two extremes. On one end sit the cloud-managed services — DataRobot, Vertex AI, Azure ML Studio — which hide the complexity behind polished interfaces but require users to ship their data into someone else's infrastructure. On the other end live the open notebooks where everything is possible and nothing is easy: Jupyter, Colab, raw Python. The space between those two poles, where a non-technical user might want to build a model without either learning Python or handing over their data, is surprisingly thin.

That gap is the question this project sets out to answer. The goal was a platform that gives a domain expert — a clinician, an analyst, a small business owner — enough visual scaffolding to build, train, and inspect a model on their own machine. Not a tutorial, not a hosted SaaS, but a real piece of software that runs locally, accepts local data, and never phones home to a vendor API.

Three workloads sit behind the same canvas: tabular ML, RAG-style generative AI, and image classification. Each one imposes a different operational profile, and fitting them into a single coherent product forced choices that were not obvious upfront. The report is organised as follows. Chapter 1 sets the project context. Chapter 2 reviews the existing landscape and the problems the new platform is meant to fix. Chapter 3 describes the methodology and the toolchain. Chapters 4 through 9 walk through the six sprints in order: the foundation sprint, then the three AI workloads back to back (ML, LLM, DL), then the collaboration layer, then the operability sprint that turned the platform from "demonstrably works" into "demonstrably operable". Chapter 10 covers testing and deployment. The General Conclusion steps back to assess what worked, what didn't, and what I would build with another quarter.

Chapter 1

General Context of the Project

Introduction

This chapter frames the problem the platform is built to address. It describes the host context, the platform itself in plain terms, the functional and non-functional requirements I committed to before writing code, the technical architecture at a high level, and the deliverables the work was evaluated against.

1.1 Project Overview and Context

1.1.1 Host context

The project was developed as an end-of-studies (PFE) project for a National Bachelor's Degree in Information Technologies, specialisation Development of Information Systems. The deliverable is a working software product plus this report. The host setting is academic rather than corporate: the supervisor is the academic advisor, the audience is the jury, and the demo machine is a personal laptop.

1.1.2 Project description — the no-code AI platform

The platform is a self-hosted web application. A signed-in user can:

- Upload tabular data (CSV, Excel), pull it from a Postgres or MySQL connector, or upload a zip of images organised by class folder.
- Inspect a profile of the dataset — missing values, skew, outliers, target imbalance, correlations — before doing anything else with it.
- Build a pipeline visually on a React Flow canvas, in one of three modes:
  - **Traditional ML** — fourteen supervised and unsupervised algorithms spanning classification, regression, clustering, and time-series forecasting.
  - **Generative AI** — index documents into a local vector store and chat with a RAG pipeline.
  - **Deep Learning** — image classification on a fixed catalogue of three CNN architectures.

- Train, evaluate, download the trained artefact, and — for image models — run live inference on a hand-uploaded test image.
- Collaborate with teammates inside a company workspace, with role-based access control, real-time canvas presence, threaded per-pipeline chat, and one-click Google Meet links.

A super-admin role sits alongside the user-facing application with its own dashboard for user management, audit logs, queue health, hardware monitoring, and a "view-as-user" impersonation flow gated by a hard five-minute TTL.

PLACEHOLDER: Landing page mockup

Figure 1.1: Landing page interface

1.2 Requirements Specification

1.2.1 Functional Requirements

- Tabular ML pipelines: dataset → train → evaluate, backed by fourteen algorithms.
- RAG pipelines: document → vector store → chat, end to end on the local box.
- DL pipelines: image dataset → CNN architecture → training loop, fully local.
- User authentication with email/password and optional TOTP-based two-factor authentication.
- Stripe-backed billing across free, solo, and company tiers, with super-admin overrides on per-user quotas.
- Real-time presence and shared editing on the pipeline canvas.
- Per-pipeline access control with five roles (Owner, Project Manager, Data Scientist, Analyst, Viewer).
- Super-admin operations: user management, audit logs, queue and hardware monitoring, user impersonation.

1.2.2 Non-Functional Requirements

- **Data sovereignty.** No outbound calls to AI vendors. Stripe is the only third-party service in the deployment graph, and it is optional — without an API key the platform still works on the free tier.
- **Modularity.** The system splits into five independent Python services so a long training run cannot block an authentication request.
- **Reproducibility.** Every dataset version is profiled, every training run is recorded, every model artefact is versioned and downloadable.

- **Hardware constraints.** The whole stack runs on a single consumer machine. My development box — a laptop with 16 GB of RAM and a GTX 1660 Super GPU — was picked on purpose, because it sits close to the floor of what someone might realistically deploy on.
- **Accessibility.** WCAG AA contrast on the default theme, with a high-contrast theme available from the navbar.
- **Internationalisation.** English and French resource bundles, with TypeScript bindings generated from the English bundle so a missing translation fails the type-check.

1.3 Technical Architecture

1.3.1 Technology Stack

- **Frontend.** React 18, Vite, TypeScript, MUI, React Flow, Plotly, i18next, Zustand.
- **Backend.** Five Flask services, Celery for CPU-bound work, Flask-SocketIO for live channels.
- **Data.** Postgres (with `pgvector`), MongoDB, Redis, TimescaleDB for metrics.
- **ML.** scikit-learn, XGBoost, LightGBM, CatBoost, H2O, Prophet, SHAP.
- **LLM.** Ollama serving Llama 3.2 3B (`Q4_K_M` quantisation), sentence-transformers (`all-MiniLM-L6-v2`) for embeddings.
- **DL.** PyTorch + torchvision, CUDA 12.1.
- **Ops.** Docker Compose, GitHub Actions, MailHog for development email capture.

1.3.2 Technical Constraints

- Single host, single GPU (6 GB VRAM).
- No cloud AI vendor in the request path — a hard non-negotiable, not a preference.
- Stack must come up with a single `make up` on a clean machine.
- CI must complete in under fifteen minutes for a full-stack change.

1.4 Deliverables and Evaluation Criteria

1.4.1 Project Deliverables

- A working web application running the three pipeline modes end to end on the demo machine.
- Five Python microservices, packaged as Docker images.
- A React + TypeScript frontend, packaged as a static build.

- A `docker-compose.yml` that brings the entire stack online with one command.
- A CI pipeline (lint + tests, parallel jobs) green on the `main` branch.
- This written report and the supporting diagrams.

1.4.2 Evaluation Criteria

- Functional coverage of the three pipeline modes.
- Soundness of the architecture (separation of concerns, explicit security boundaries, reproducible deployment).
- Code quality (typing, linting, test coverage on the load-bearing paths).
- Quality of the report and the live demo.

1.5 Interface Mockups

PLACEHOLDER: Login + 2FA screen

Figure 1.2: Authentication interface (email/password + optional TOTP)

PLACEHOLDER: Home / Dashboard screen

Figure 1.3: Home page interface for a signed-in user

PLACEHOLDER: Admin operations dashboard

Figure 1.4: Admin dashboard interface

1.6 Project Duration

The project ran across six sprints of roughly two to three weeks each, with a runnable demo and a self-contained git tag at the end of every sprint. The timeline, with start and end dates, is captured in the Gantt chart in Chapter 3.

Conclusion

The chapter framed the problem the rest of the report addresses — how to make ML approachable without sacrificing data sovereignty — and sketched the platform that emerged in response. The next chapter reviews the existing landscape to explain why the question is harder to answer with off-the-shelf tools than one might initially assume.

Chapter 2

Preliminary Study

Introduction

Before describing how the platform was built, it helps to look at why nothing already on the market quite fits the problem. The no-code ML space is crowded, but the products in it cluster into families that all share the same blind spot.

2.1 Current Situation

Three categories of product dominate the space:

- **Cloud-native AutoML platforms** — DataRobot, H2O Driverless AI, Azure ML Studio, Google Vertex AI. Highly polished, exhaustive algorithm catalogues, almost invariably SaaS-only. Their privacy story ends at "trust our infrastructure". For some buyers that is fine; for regulated industries it isn't.
- **Open-source notebooks** — Jupyter, Google Colab, DataLab. Powerful for experts, but every workflow eventually bottoms out in raw Python. That is exactly the friction the no-code category was invented to remove.
- **Visual ETL tools** — KNIME, Alteryx, RapidMiner. Closer in spirit to this project, but their centre of gravity is data preparation rather than model training, and their LLM integrations tend to route requests back through OpenAI.

PLACEHOLDER: Comparison table of existing solutions

Figure 2.1: Positioning of existing no-code ML solutions

2.2 Observed Problems

2.2.1 Forced data exfiltration

A clinician analysing patient records or a bank analyst classifying transactions cannot lawfully upload that data to a vendor bucket. The problem isn't contractual — it is regulatory (GDPR, HIPAA, local banking rules). Every SaaS AutoML platform in the previous section assumes the user is fine with that upload. They are not.

2.2.2 Cloud LLM dependence in generative-AI tooling

Visual tools that have added an "AI" panel in the last two years almost always route the request through OpenAI or a similar API. The panel is bolted on, not local. For a buyer who can't ship the prompt out either, the panel is unusable.

2.2.3 The notebook cliff

Notebooks solve the locality problem cleanly. They fail on the no-code one. A domain expert who can read a confusion matrix but cannot write `from sklearn.ensemble import RandomForestClassifier` hits a wall on line one of the first cell. Closing that gap is the whole reason a visual canvas exists in the first place.

2.2.4 Hardware ambiguity

The few self-hostable products that do exist (KubeFlow, MLflow on local Kubernetes) assume the operator has a Kubernetes cluster sitting around. That is not the operating environment of a small team that has a workstation and an ambition.

2.3 Objectives of the New System

The platform I built sets out to satisfy four objectives at once:

1. **Locality.** Every model server, vector store, queue, and database runs inside the same Docker Compose stack on the user's own machine.
2. **Coverage.** Three workloads — tabular ML, RAG-style generative AI, and image classification — share the same canvas. The user does not need a different product for each.
3. **Approachability.** A canvas-based UI that a non-programmer can drive end to end, with sensible per-node validation and informative error messages on the unhappy path.
4. **Honest hardware story.** The reference deployment is a laptop with 16 GB of RAM and a GTX 1660 Super (6 GB VRAM). The platform must run on it, not just compile on it.

Two technical shifts make the fourth objective realistic today and would not have made it realistic two years ago.

1. **Aggressive LLM quantisation.** Four-bit quantisation (`Q4_K_M`) shrinks a three-billion-parameter model from roughly 12 GB of weights down to about 2 GB, while preserving most of the original quality on conversational tasks. The trade-off is real but, for chat, it isn't fatal.
2. **First-class GPU inference runtimes for consumer hardware** (Ollama, llama.cpp, vLLM). A 1660 Super, with 6 GB of VRAM and 30 TFLOPS of FP16 throughput, can serve roughly 30 tokens per second on a 3B model — enough latency budget for an interactive RAG chat.

PLACEHOLDER: Quantisation → VRAM mapping

Figure 2.2: Mapping of quantisation level to VRAM footprint for a 3B-parameter model

Conclusion

Cloud platforms solve the wrong half of the problem, notebooks demand too much of the user, and visual ETL tools are not aimed at model training. The platform described in the rest of this report fills the gap deliberately: it is local by construction, broad enough to cover the three AI workloads that matter today, and modest enough in its hardware appetite to actually run on a laptop.

Chapter 3

Methodology and Working Environment

Introduction

Six sprints, one person, three AI workloads on a single GPU — the project's main risk was not technical. It was scope management. This chapter explains the methodology I used to keep that risk in check, the toolchain that anchored the build, and the layout of the services those choices produced.

3.1 Comparison of Project Management Methodologies

I considered three styles before settling.

- **Waterfall.** Plan everything upfront, build everything in order, demo at the end. Wrong shape for an ML project where the cost of a feature is genuinely unpredictable (Prophet's CmdStan dependency added two hours to a "trivial" algorithm).
- **Pure Kanban.** Continuous flow, no sprint boundary. Tempting for a solo project, but it removes the checkpoints I needed to demo to my supervisor.
- **Scrum-lite (chosen).** Short sprints, a runnable demo at the end of each, a self-contained git tag, a retrospective note for myself. Enough ceremony to enforce scope discipline; not so much that ceremony eats the sprint.

3.2 The Choice of Agile Methodology

Two reasons pushed me toward Agile. First, ML capacity planning is unpredictable. A feature that looks easy on paper can hide a compile-from-source dependency, and Agile's sprint boundary absorbs that kind of slip. Second, I needed dated checkpoints I could demo without hand-waving. Without sprint structure I would have ended up with a single "everything works at once" deliverable, no story to tell along the way, and no fallback position if a late feature fell through.

3.3 Scrum Methodology

PLACEHOLDER: Scrum framework diagram

Figure 3.1: The Scrum framework: roles, artefacts, events

3.3.1 Scrum Roles

In a one-person team the roles all collapse onto the same person. What does not collapse is the *discipline* of switching hats. When planning, I am the Product Owner — I write user stories and prioritise. When building, I am the Developer. When reviewing the sprint, I am the Scrum Master — what went wrong, what would I do differently. Treating those as three roles, even when they live in one head, kept me honest.

3.3.2 Scrum Artefacts

- **Product Backlog.** A single Markdown file under `docs/backlog.md`, grouped by sprint.
- **Sprint Backlog.** The slice of the product backlog promised for the current sprint, lifted into the sprint's section.
- **Increment.** The git tag at the end of the sprint, plus the demo recording.

3.3.3 Scrum Events

- **Sprint Planning.** Half a day at the start of each sprint. Carry-over from the previous sprint plus a fresh slice of the backlog.
- **Daily check-in.** A short written log entry, not a meeting.
- **Sprint Review.** A live demo with my supervisor at sprint end.
- **Sprint Retrospective.** A retro note in the same Markdown backlog, under the sprint heading.

3.4 Product Backlog

The six sprints below each ended with a runnable demo and a self-contained git tag.

- **Sprint 1 — Authentication, Gateway, Data Ingestion.** Foundational service mesh.
- **Sprint 2 — Visual Pipeline Editor and Tabular ML.** Canvas, fourteen algorithms, SHAP, model export.
- **Sprint 3 — Local Generative AI and RAG.** Ollama, `pgvector`, streamed chat.

- **Sprint 4 — Deep Learning for Image Classification.** PyTorch service, three CNNs, the VRAM guard.
- **Sprint 5 — Multi-tenancy and Real-Time Collaboration.** Project ACL, cursors, threaded chat, Meet links.
- **Sprint 6 — Dashboard, i18n, Accessibility, Admin Console.** Operability and polish.

3.5 Project Planning

PLACEHOLDER: Gantt chart

Figure 3.2: Gantt chart for the six-sprint plan

The Gantt chart in the figure above tracks planned versus actual duration. Two sprints slipped by roughly a week each — Sprint 2 because the live training visualisation had a Plotly autoscale bug that took a day to find, and Sprint 4 because the static VRAM estimate needed an empirical recalibration pass on real hardware.

3.6 Technologies

- **Frontend.** React 18 + Vite + TypeScript + MUI. React for the ecosystem, Vite because cold starts under `create-react-app` were embarrassingly slow, MUI because it ships accessible components out of the box — a point that matters for the company-tier sale.
- **Backend.** Flask + Celery + Flask-SocketIO. Flask was the lowest-common-denominator choice for a microservice. FastAPI would have given me typed routes for free, but every template I started from assumed Flask, and going back through five rewrites just for typing wasn't a fight I wanted to have. Celery handles every CPU-bound task: profiling, preprocessing, H2O training, RAG ingestion, image extraction, and PyTorch training each live on their own queue.
- **Postgres + pgvector for relational and vector data.** Co-locating the vector store with the auth schema let me avoid spinning up a sixth backing service. `pgvector`'s IVFFlat index is more than enough for the few hundred thousand chunks that a single-tenant local deployment is ever likely to hold.
- **MongoDB for everything schemaless.** Datasets, pipelines, model versions, task results — anything whose shape evolves between sprints lives in Mongo.
- **Redis for queue + cache + socket message bus.** Three logical databases on the same instance: DB 0 for Celery, DB 1 for Celery results, DB 2 for the SocketIO message queue.
- **Ollama for local LLM inference.** The alternative was calling `llama.cpp` directly. That would have given me finer control at the cost of building my own

HTTP shim around it; Ollama's REST API is good enough that the trade wasn't close.

- **PyTorch + torchvision for image classification.** The CUDA wheels are downloaded from PyTorch's CUDA 12.1 index inside the Dockerfile so the host only needs the NVIDIA driver plus `nvidia-container-toolkit`.

3.7 Architecture

PLACEHOLDER: Microservice topology diagram

Figure 3.3: High-level microservice topology

- `api-gateway` (port 8000) — decodes JWTs, injects `X-User-Id` / `X-User-Tier` / `X-User-Role` headers, proxies to the upstream services. Also hosts the SocketIO server for chat and training presence.
- `auth-service` (port 8001) — user accounts, billing, admin operations, project and company ACL.
- `data-ingestion-service` (port 8002) — file uploads, SQL and S3 connectors, profiling, preprocessing, RAG ingestion, image-dataset extraction.
- `ml-training-service` (port 8003) — the fourteen tabular algorithms, model registry, RAG chat orchestration.
- `metrics-service` (port 8004) — TimescaleDB-backed hardware-telemetry collection.
- `dl-training-service` (port 8005) — PyTorch image classification.

A GitHub Actions workflow runs four jobs in parallel on every push: backend lint (`black`, `isort`, `flake8`), frontend lint (`eslint`, `tsc -noEmit`), backend tests (a Postgres + Mongo + Redis stack is spun up as services, then `pytest` runs against each microservice), and frontend tests (`vitest` with coverage upload). End-to-end CI runs in roughly six minutes for a frontend-only change, nine minutes for a backend change, and fourteen minutes when both sides move at once.

Conclusion

The methodology and toolchain held throughout the six sprints. Some choices — Flask in particular — are ones I'd revisit if starting over, but they got me to a working system without rework. The next chapter starts the sprint walkthrough.

Chapter 4

Sprint 1: Authentication, Gateway, and Data Ingestion

Introduction

Sprint 1 was the foundation sprint. Nothing glamorous — no canvases, no models — but the design choices made in these two weeks shaped every later sprint. The gateway pattern and the shared-volume convention introduced here are the quiet load-bearing decisions of the whole project.

4.1 Sprint Backlog

- Stateless JWT authentication with refresh-token rotation.
- Redis-backed token revocation list.
- Role-based access control (super-admin, tenant roles).
- API gateway that strips client-supplied identity headers and injects authoritative ones from the decoded JWT.
- Optional TOTP two-factor authentication.
- Per-IP and per-user rate limiting at the gateway.
- `data-ingestion-service`: file uploads, SQL connectors, asynchronous profiling.
- Mongo-backed dataset metadata.
- Parquet-based preprocessing output (train/val/test split).

4.2 Design

PLACEHOLDER: Use case diagram for Sprint 1

Figure 4.1: Use case diagram — authentication and ingestion

PLACEHOLDER: Login + JWT issuance sequence

Figure 4.2: Sequence diagram — login + refresh-token rotation

PLACEHOLDER: Asynchronous upload sequence

Figure 4.3: Sequence diagram — upload + asynchronous profiling

4.2.1 Why an opaque-but-decoded gateway

Two reasonable architectures were on the table:

1. Have the gateway hit `auth-service` on every request to validate the JWT.
2. Have the gateway hold the same `JWT_SECRET_KEY` as `auth-service` and decode the token in-process.

I went with the second, and I'd make the same call again. The gateway is on the hot path; a synchronous HTTP round-trip per request adds latency that compounds badly under load. The price is that the gateway needs the same signing secret as the auth service — which, in a shared environment-file deployment, is a non-issue. In a horizontally scaled production setup it would be more involved; for the demo target it's the right trade.

4.2.2 Header-injection rules

The gateway strips any client-supplied `X-User-*`, `X-Company-*`, or `X-Project-*` header before forwarding, then re-adds them from the decoded JWT. This is, without exaggeration, the most important single security invariant in the whole system. **Downstream services trust `X-User-Id` unconditionally, which means an attacker who can supply that header without going through the gateway can impersonate anyone.** The strip-then-inject pattern is deliberate, well tested, and not negotiable.

4.2.3 The shared-volume convention

My first instinct on the ingestion side was to have the API stream files to the worker over the network. I started building that, then realised halfway in that my Docker Compose stack already had a named volume (`uploaded_files`) that both the API container and the worker container could mount. Mounting it in both places collapsed the streaming-upload protocol into a one-line write: the API saves to `/uploads/<id>.csv`, the worker reads from the same path. That pattern then became the convention for every binary artefact in the system — trained models, RAG documents, image-dataset extracts.

4.2.4 Why MongoDB for dataset metadata

A dataset's profile is fundamentally schemaless. A CSV with two numeric columns produces a profile that looks nothing like the profile of a CSV with thirty mixed-type columns including a target. I spent the better part of a day considering a Postgres JSONB column for this and ultimately preferred Mongo's storage model. It's lossless, it's queryable enough for the use case, and it doesn't impose a schema migration every time I want to add a new statistic.

PLACEHOLDER: Class diagram for Sprint 1

Figure 4.4: Class diagram — auth-service + data-ingestion-service

4.3 Implementation

```python
def _forward(upstream_base, path, require_auth=True, timeout=30):
    if require_auth:
        try:
            verify_jwt_in_request()
            claims = get_jwt()
            g.user_id = get_jwt_identity()
            g.user_role = claims.get("role", "")
            g.user_tier = claims.get("tier", "free")
        except Exception as e:
            return jsonify({"error": "unauthorized", "message": str(e)
    }), 401

    # Strip every client-supplied identity header before forwarding.
    _STRIP_PREFIXES = ("x-user-", "x-company-", "x-project-")
    headers = {
        k: v for k, v in request.headers
        if k.lower() not in _HOP_BY_HOP
        and k.lower() != "host"
        and not k.lower().startswith(_STRIP_PREFIXES)
    }
    # Re-inject the authoritative ones from the decoded JWT.
    if require_auth or hasattr(g, "user_id"):
        headers.update(get_forwarded_headers())

    resp = requests.request(
        method=request.method, url=f"{upstream_base.rstrip('/')}{path}"
    ,
        headers=headers, data=request.get_data(), stream=True, timeout=
    timeout,
    )
    return _stream_response(resp)
```

Listing 4.1: Gateway proxy with header injection (`api-gateway/app/routes/proxy.py`)

```python
@celery.task(name="app.tasks.profiling.profile_dataset", bind=True)
def profile_dataset(self, dataset_id: str):
    client = MongoClient(os.environ["MONGO_URL"])
    db = client[os.environ.get("MONGO_DB", "nocode_ingestion")]
    datasets = db["datasets"]
    task_results = db["task_results"]

    task_results.update_one(
        {"task_id": self.request.id},
        {"$set": {"status": "running", "progress_pct": 0}},
        upsert=True,
    )
    doc = datasets.find_one({"dataset_id": dataset_id})
    df = load_dataframe(doc["file_path"])

    summary = compute_profile_summary(df, target_column=doc.get("
```

```
    target_column"))

    datasets.update_one(
        {"dataset_id": dataset_id},
        {"$set": {
            "status": "ready",
            "profiling_summary": summary,
            "row_count": len(df),
            "column_count": len(df.columns),
        }},
    )
    task_results.update_one(
        {"task_id": self.request.id},
        {"$set": {"status": "success", "progress_pct": 100}},
    )
```

Listing 4.2: Profiling task (`data-ingestion-service/app/tasks/profiling.py`)

4.3.1 Security posture

- Access tokens have a fifteen-minute TTL. Refresh tokens live as HttpOnly cookies and rotate on every refresh; the previous token's JTI lands in the Redis blacklist so a stolen refresh token can actually be revoked.
- Passwords use `bcrypt` at cost 12. Refresh tokens themselves are stored as SHA-256 hashes in Postgres — the database-breach scenario yields a hash, not a usable token.
- TOTP-based two-factor authentication is optional per user; super-admins can require it per-tier from the admin panel.
- Per-IP and per-user rate limits sit at the gateway. The per-user limit defaults to 600 req/min in development, deliberately loose because React 18 Strict Mode double-invokes `useEffect` and a tighter cap was producing spurious 429s.

4.3.2 SQL connectors and Fernet encryption

Users can configure a Postgres or MySQL connector inside the platform; the credentials are encrypted with a Fernet key (AES-128-CBC plus HMAC) held in the service's environment, then stored in Mongo. The same key is reused for any other secret the ingestion service needs to hold, and rotating it is a documented operations procedure rather than a fire-fight. SQL queries themselves run through SQLAlchemy with parameter binding, which makes SQL injection structurally impossible rather than merely unlikely.

PLACEHOLDER: Login screen

Figure 4.5: Login screen with optional TOTP field

PLACEHOLDER: Connector wizard

Figure 4.6: SQL connector wizard — credentials entered, encrypted at rest

4.4 Unit Tests

The auth-service test suite sits above 80% line coverage on the authentication path and the admin operations. Notable cases:

- Forged `X-User-Id` header: the request goes through the gateway, the forged header is stripped, the authoritative one from the JWT replaces it. Asserted on a synthetic upstream that echoes back the headers it saw.
- Refresh-token replay: a refresh token is used once, then replayed. The second use is rejected by the blacklist.
- Expired access token: the gateway returns 401 with the documented error envelope.
- Rate limit: 601 requests in one minute, the 601st returns 429 with a `Retry-After` header.

The ingestion suite uses `mongomock` so it does not need a real Mongo to run. It exercises the upload-then-profile path end-to-end with a small fixture CSV and asserts the profile shape.

Conclusion

By the end of Sprint 1 the platform had end-to-end authentication, a working refresh-token loop, a gateway pattern the next five sprints would build on, and an ingestion service that could profile arbitrarily messy CSVs asynchronously. The authoritative-headers convention turned out to be one of the project's most quietly useful design decisions: every subsequent service trusts `X-User-Id` without a second thought, because the gateway is the only path along which it can be set.

Chapter 5

Sprint 2: Visual Pipeline Editor and Tabular Machine Learning

Introduction

Sprint 2 was the largest single sprint of the project. It pulled together four big pieces — the visual pipeline editor, the `ml-training-service` with its fourteen tabular algorithms, the live metric visualisations, and the post-training surface (SHAP explainability, model export) — into a coherent flow the user could actually follow from canvas to model.

5.1 Sprint Backlog

- React Flow canvas with custom node components.
- Pipeline schema (Mongo document with nodes + edges).
- `ml-training-service` hosting fourteen tabular algorithms.
- `BaseMLModel` abstraction and an algorithm-agnostic Celery training task.
- Model registry with versioning.
- Live metric streaming over SocketIO.
- SHAP explainability for supervised runs.
- Confusion matrices, residual plots.
- Model export (joblib + generated FastAPI inference script).

5.2 Design

PLACEHOLDER: Use case diagram for Sprint 2

Figure 5.1: Use case diagram — canvas + tabular ML

PLACEHOLDER: Class diagram for Sprint 2

Figure 5.2: Class diagram — BaseMLModel hierarchy + model registry

PLACEHOLDER: Pipeline editor screenshot — ML mode

Figure 5.3: Pipeline editor in tabular ML mode

5.2.1 Graph schema

A pipeline is stored in Mongo as a document with two arrays:

```
{
  "pipeline_id": "...",
  "user_id": "...",
  "type": "ml" | "rag" | "dl",
  "nodes": [
    { "node_id": "n1", "type": "dataset", "position": {"x": 60, "y": 120},
      "data": { "dataset_id": "..." } },
    { "node_id": "n2", "type": "train", "position": {"x": 320, "y": 120},
      "data": { "algorithm": "xgboost", "task_type": "classification",
                "hyperparams": {...}, "target_column": "..." } },
    { "node_id": "n3", "type": "evaluate", "position": {"x": 580, "y": 120},
      "data": {} }
  ],
  "edges": [
    { "source": "n1", "target": "n2" },
    { "source": "n2", "target": "n3" }
  ],
  "status": "draft"|"running"|"done"|"error"
}
```

Listing 5.1: Pipeline document shape

5.2.2 The `BaseMLModel` abstraction

Fourteen algorithms with one orchestrator implies an abstraction. Rather than over-engineer it, I went with a deliberately small abstract base class: each algorithm subclass exposes `train`, `predict`, and `evaluate` with a shared signature, whether it's wrapping scikit-learn, XGBoost, LightGBM, CatBoost, or Prophet. The training Celery task is algorithm-agnostic — it constructs a model from a registry, trains it, asks it for metrics, and serialises the estimator. Adding a new algorithm is, in practice, a one-file change.

```
class BaseMLModel(ABC):
    def __init__(self, hyperparams: dict[str, Any]):
        self.hyperparams = hyperparams
        self._estimator = None

    @abstractmethod
    def train(self, X_train, y_train=None) -> None: ...
    @abstractmethod
    def predict(self, X) -> Any: ...
    @abstractmethod
```

```
    def evaluate(self, X_test, y_test=None) -> dict[str, Any]: ...

    @property
    def estimator(self):
        return self._estimator
```

Listing 5.2: Algorithm-agnostic training core

The registry mapping algorithm strings to subclasses lives in `ml-training-service/app/services/`

5.2.3 H2O for the heavy algorithms

For XGBoost, GBM, and Random Forest, I used H2O-3 instead of the scikit-learn implementations. H2O's algorithms run faster on tabular data, and they expose feature importance and SHAP values out of the box — which, on its own, removed an entire subtask from the explainability work. Not every dependency pays for itself; this one did.

5.3 Implementation

5.3.1 Frontend canvas

PLACEHOLDER: Node component anatomy

Figure 5.4: Anatomy of a canvas node component — header, body, ports, validation badge

The canvas is a React Flow instance with custom node components. Each node type validates itself against the surrounding graph: the `Train` node shows an error badge if no `Dataset` node is upstream; the `Evaluate` node shows a warning if no training run has produced a version yet. The validation function is pure — `(node, edges, allNodes)` → `{status, message}` — which keeps it trivially testable.

5.3.2 Live training visualisation

Training emits per-step metric points over the SocketIO message queue. The `LiveTrainingChart` component subscribes to the pipeline's room (`pipeline_<id>`) on the `/training` namespace and plots the points on a Plotly scatter. The Y-axis is auto-scaled because progress percentage (0–100) and metric values (typically 0–1) live on different ranges — progress on the primary axis, metrics on the secondary. A small thing, but the chart was unreadable until I got that right.

PLACEHOLDER: Live training chart

Figure 5.5: Live training chart — progress on the primary axis, metric on the secondary

5.3.3 SHAP and the explainability surface

Every supervised training run computes SHAP values over a sampled subset of the test set. The bar chart of mean absolute SHAP values surfaces in the `Evaluate` node's results panel. For tree-based models — XGBoost, Random Forest, GBM, LightGBM, CatBoost — the SHAP computation uses the optimised `TreeExplainer` path, which is roughly two orders of magnitude faster than `KernelExplainer`. On a 5,000-row test set, that's the difference between SHAP being a feature and SHAP being a wait.

PLACEHOLDER: SHAP feature importance chart

Figure 5.6: SHAP feature importance bar chart

5.3.4 Model export

The export step packages the trained `.joblib` artefact along with a generated FastAPI inference script. The user downloads a zip they can drop onto any container runtime to serve their model without the platform at all. That property — being able to leave — mattered to me on principle. A no-code platform that locks you in isn't really removing friction; it's relocating it.

5.3.5 Why `joblib` and not `pickle`

`joblib` was the deliberate choice for the tabular path because its custom pickler is optimised for objects containing large NumPy arrays — which every scikit-learn estimator does — and can memory-map them on load. For a Random Forest of 500 trees the size difference was roughly 8× in my measurements; the load-time difference was larger. For the deep-learning path in Sprint 4 I used `torch.save` on a `state_dict()` instead. Different ecosystem, different convention.

5.4 Unit Tests

- Each `BaseMLModel` subclass has a happy-path test on a tiny fixture dataset (50 rows, four columns, a binary target). Assertions on output shape and on the presence of the documented metrics keys.
- The algorithm-factory is exercised by parametrised tests — one per algorithm string — that construct the model, train, predict, and inspect the result envelope.
- Node validation: parametrised tests over (node-type, upstream-edges, expected-status) tuples.
- Export bundle: the generated zip is unzipped to a `tempdir` and the FastAPI script is imported to confirm it parses. The actual inference call is exercised by an end-to-end test in the deployment chapter.

Conclusion

Sprint 2 was where the project first felt like a real product. A user could open the canvas, drop three nodes, hit Run, and watch a model train in front of them. The `BaseMLModel` abstraction introduced here later absorbed the deep-learning extension in Sprint 4 without disturbing the orchestration code, and the SHAP + export surface closed the loop between training a model and being able to do something with it.

Chapter 6

Sprint 3: Local Generative AI and Retrieval-Augmented Chat

Introduction

This was the most technically ambitious sprint of the project. A fully local Retrieval-Augmented Generation pipeline meant fitting an embedding model, a vector store, and an LLM inside the same Docker stack as the rest of the platform — and making the chat experience responsive enough that users wouldn't notice they were running it on a consumer GPU.

6.1 Sprint Backlog

- Document ingestion pipeline (PDF / DOCX / text).
- Local embedding via `sentence-transformers`.
- Vector store in `pgvector` (IVFFlat index).
- Ollama integration serving Llama 3.2 3B (`Q4_K_M`).
- Chat orchestration: embed question, retrieve top-k chunks, build prompt, stream the answer back.
- Streamed UI: tokens render as they arrive over SocketIO.
- RAG-mode toggle on the canvas with three dedicated node types (`DocumentNode`, `VectorStoreNode`, `ChatNode`).

6.2 Design

PLACEHOLDER: Use case diagram for Sprint 3

Figure 6.1: Use case diagram — RAG pipeline

The component picks:

PLACEHOLDER: RAG pipeline sequence diagram

Figure 6.2: Sequence diagram — document → embed → store → chat

PLACEHOLDER: Class diagram for Sprint 3

Figure 6.3: Class diagram — RAG ingestion + chat orchestrator

- `sentence-transformers/all-MiniLM-L6-v2` for embeddings — 384 dimensions, around 22 MB on disk, fast enough on CPU that I never had to think about GPU scheduling for it.
- `pgvector` for the vector store, with an IVFFlat index. Co-locating it in the auth-service Postgres instance avoided a sixth backing service.
- **Ollama running** `llama3.2:3b` as the LLM engine. The `Q4_K_M` quantisation fits in roughly 2 GB of VRAM, which leaves headroom on the 6 GB demo GPU for embedding, canvas, and the training queue to share the same device without stepping on each other.

6.3 Implementation

```python
@celery.task(name="app.tasks.rag_ingest.process_rag_document", bind=True)
def process_rag_document(self, document_id, pipeline_id, file_path):
    from langchain_text_splitters import RecursiveCharacterTextSplitter
    from langchain_community.document_loaders import PyPDFLoader

    docs = PyPDFLoader(file_path).load()
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=500, chunk_overlap=50, length_function=len,
    )
    split_docs = splitter.split_documents(docs)
    chunk_texts = [d.page_content for d in split_docs if d.page_content.strip()]

    # Local embedding via sentence-transformers -- no network call.
    vectors = embed_texts(chunk_texts)
    insert_chunks(
        pipeline_id=pipeline_id,
        document_id=document_id,
        chunks=chunk_texts,
        embeddings=vectors,
    )
```

Listing 6.1: Document ingestion task (`ml-training-service/app/tasks/rag_ingest.py`)

6.3.1 Streamed chat

The chat response streams token by token over SocketIO. The `ml-training-service` plays producer: it embeds the question, queries pgvector, builds the prompt, and for-

wards Ollama's `stream=true` chunks straight into the `/chat/<pipeline_id>` room. The frontend appends them to the current assistant message as they arrive. Watching that for the first time, on a laptop, with no network involved, was one of the more satisfying moments of the project.

PLACEHOLDER: RAG chat screenshot

Figure 6.4: RAG chat panel — tokens stream in as the model emits them

6.3.2 Chunk sizing trade-offs

Five hundred characters with a fifty-character overlap is the default chunking. I tried shorter chunks (256, 128) and longer ones (1024). The short variants improved retrieval precision on short-answer questions and ruined it for anything that needed multi-sentence context; the long variants did the opposite. Five hundred sat in the middle. It is a defensible default, not a universal answer; the project backlog has a note to expose chunk size as a per-pipeline setting.

6.4 Unit Tests

- Chunking determinism — the same input file produces the same chunks across runs.
- Embedding shape — 384 dimensions, normalised.
- Retrieval top-k on a deterministic five-chunk corpus with hand-labelled relevance.
- Chat orchestrator: Ollama is mocked at the HTTP boundary, the orchestrator is asserted to emit the right SocketIO events in the right order.

The Ollama process itself is not exercised in unit tests — it is covered by the smoke-test script in the deployment chapter.

Conclusion

Sprint 3 proved the local-generative-AI thesis to my own satisfaction. The chat is not as fast as GPT-4 over the wire, and it never will be on this hardware, but it's interactive enough to be useful and it keeps the user's documents on their machine. For the regulated industries the platform was designed for, that trade-off is exactly the one they're looking to make.

Chapter 7

Sprint 4: Deep Learning for Image Classification

Introduction

This sprint added the third pipeline mode. Deep learning on the demo hardware is a constrained problem — 6 GB of VRAM does not forgive carelessness — so the design started from the constraint and worked backwards. The result is a small, opinionated catalogue rather than a kitchen sink, and that opinionatedness is the feature.

7.1 Sprint Backlog

- New microservice: `dl-training-service`.
- Three CNN architectures (LeNet, ResNet-9, MobileNet-V3-Small).
- Image-dataset ingestion (zip of `<class>/<file>`).
- Static VRAM estimator (the "VRAM guard").
- Tier-aware ceilings on epochs and batch size.
- Live training visualisation (reused from Sprint 2).
- Inline single-image inference (the "Try it" panel).
- DL-mode canvas toggle with three new node components.

7.2 Design

PLACEHOLDER: Use case diagram for Sprint 4

Figure 7.1: Use case diagram — DL pipeline

PLACEHOLDER: DL service architecture

Figure 7.2: Architecture — `dl-training-service`, Celery worker, GPU allocation

PLACEHOLDER: Class diagram for Sprint 4

Figure 7.3: Class diagram — architecture catalogue + VRAM guard

7.2.1 Scope decisions

What's in:

- Three architectures: LeNet (smoke-test scale), a custom ResNet-9 (*tiny_ resnet*), and torchvision's MobileNet-V3-Small with optional ImageNet transfer learning.
- Image-dataset upload: a zip of `<class>/<file>` folders, extracted into the canonical `ImageFolder` layout that PyTorch's data loaders expect.
- Live training visualisation reused from the tabular path (the `training_epoch` socket event fans out into train/val loss and val-accuracy series).
- Single-image inference (the "Try it" panel) immediately after training completes.

What's deliberately out:

- Freeform layer-builder DAG — too easy to OOM on stage.
- LLM fine-tuning — won't fit inside 6 GB.
- Object detection / segmentation — different label format, different demo.
- ONNX export — a one-line follow-up if needed, not in scope.
- Multi-GPU / distributed training.

Drawing the line clearly was as important as anything I built. A no-code deep-learning interface that lets the user wire up an architecture that doesn't fit is, in practice, a debugger they didn't ask for.

7.2.2 Why a separate microservice

Two reasons:

1. `torch` plus `torchvision` add about 2 GB of dependencies that the existing H2O-based `ml-training-service` doesn't need. Co-locating them would have doubled the size of an image that rebuilds on every CI run.
2. GPU access is per-container in Docker. The DL service declares a `deploy.resources.reservatio` block; the H2O service runs on CPU. Mixing them in one container would either deny GPU to the DL workload or force the H2O workload onto a GPU it can't use.

The service has its own Celery queue (`dl_training`) so a 20-epoch CIFAR run cannot starve the fast tabular queue.

7.3 Implementation

7.3.1 The architecture catalogue

- **LeNet** — roughly 60 k parameters. Smoke-test scale; trains in seconds on Fashion-MNIST-class data.
- **Tiny ResNet (ResNet-9)** — roughly 2.7 M parameters. Two residual blocks, channels 64 → 128 → 256. Hits around 90% accuracy on CIFAR-10 in five epochs at batch 64, input 64 px.
- **MobileNet-V3-Small** — roughly 2.5 M parameters. Loaded from torchvision with optional ImageNet weights. The transfer-learning toggle is the single biggest UX win for the demo — a fresh user's 200-image custom dataset can hit 80%+ accuracy in two minutes, where random-init would take twenty.

7.3.2 The VRAM guard

The most subtle piece of Sprint 4 is the static memory estimator:

```
def estimate(arch: str, input_size: int, batch_size: int) -> Estimate:
    weights = params_mb(arch)
    optimizer = 3.0 * weights              # weights + grads + 2 Adam
    moments
    activations = (
        _ACTIVATION_PER_PIXEL_MB[arch]
        * (input_size * input_size)
        * batch_size
    )
    total = weights + optimizer + activations + _CUDA_RUNTIME_MB
    return Estimate(weights_mb=weights, optimizer_mb=optimizer,
                    activations_mb=activations,
                    runtime_mb=_CUDA_RUNTIME_MB, total_mb=total)
```

Listing 7.1: Static VRAM estimate (`dl-training-service/app/services/vram_guard.py`)

The route refuses any `/dl/train` request whose estimate exceeds the user's tier-resolved budget minus a 1 GB headroom for the CUDA runtime and allocator fragmentation. This is the single most important demo-safety net in the system: a wrong batch-size press cannot OOM the GPU on stage.

The constants in `_ACTIVATION_PER_PIXEL_MB` are conservative empirical estimates. The unit-test suite enforces that every `(arch, input_size, batch_size)` triple in the documented catalogue fits on a 6 GB card after headroom, but it does not pin the absolute numbers — those drift slightly between GPU generations and need recalibration on first deploy.

7.3.3 Tier-aware ceilings

Sprint 4 also added two new per-tier columns to the `subscriptions` table: `max_dl_epochs` and `max_dl_batch_size`. The defaults:

- **Free** — 5 epochs, batch 32.

- **Solo** — 20 epochs, batch 64.
- **Company** — 50 epochs, batch 64.

The defence-in-depth chain runs:

1. The frontend's `DLTrainNode` reads the user's `limits` from `/users/me` and clamps the slider `max` accordingly — a free-tier user literally cannot select 20 epochs from the UI.
2. The route enforces the tier ceiling regardless: if a custom client posts 50 epochs to `/dl/train` on a free-tier token, the route returns `400 epochs_over_tier_limit`.
3. The `vram_guard` then checks the static memory estimate against the tier's VRAM budget.
4. The service-wide `HARD_MAX_EPOCHS` and `HARD_MAX_BATCH_SIZE` caps in `config.py` sit as an absolute ceiling that no tier override can lift.

Four layers may sound paranoid, but each one catches a different class of mistake — accidental, malicious, oversized, or pathological — and none of them is expensive.

7.3.4 Frontend integration

PLACEHOLDER: Pipeline editor screenshot — DL mode

Figure 7.4: Pipeline editor in deep-learning mode

The canvas gained a third toggle button ("Deep Learning"), three new node components (`ImageDatasetNode`, `CNNArchNode`, `DLTrainNode`), and a fourth preset (`dlStarter`). The three-family lock logic was extended to lock all three families against each other once any node of a given family lands on the canvas.

The `ImageDatasetNode` body shows class chips, total-image count, and a deterministically seeded thumbnail strip (three images per class, pulled from `/datasets/<id>/image-preview` as base64-encoded JPEGs). The `CNNArchNode` surfaces a `pretrained` toggle that disables itself when the chosen architecture has no canonical ImageNet checkpoint — the user sees *why* the option is unavailable, rather than having a control silently disappear, which is the kind of small UX detail that tends to define whether people trust the interface.

PLACEHOLDER: ImageDatasetNode close-up

Figure 7.5: ImageDatasetNode — class chips, image count, thumbnail strip

7.3.5 Live progress and inference

The training Celery task emits a `training_epoch` socket event per epoch with `{epoch, train_loss, val_loss, val_acc}`. The `LiveTrainingChart` component fans that single event into three Plotly series, all sharing the same epoch-indexed X-axis.

When training completes, a `DLPredictPanel` component renders inline, immediately under the success alert. The user drops an image, the panel POSTs it to `/dl/predict/<version_id>`, and the top-five softmax probabilities render as ranked horizontal bars.

PLACEHOLDER: DLPredictPanel screenshot

Figure 7.6: Inline DL prediction panel — top-five softmax probabilities

7.3.6 Demo path end-to-end

For the live demo, the path is:

1. `python scripts/seed_demo_image_dataset.py /tmp/demo.zip` — generates a 30-image, 3-class synthetic dataset (red circles, green squares, blue triangles).
2. Upload the zip from the Data page.
3. Drop the `dlStarter` preset onto a fresh pipeline.
4. Click Run. Roughly 30 seconds later, the predict panel renders.
5. Drop an image — one of the synthetic ones, or a real one from a phone — and inference returns in roughly 5 ms on CPU.

A separate `scripts/dl_smoke.sh` script exercises the entire chain non-interactively: gateway reachability, GPU visibility, the oversize-refusal path, the demo run, and the polling loop. Having that script saved me on more than one occasion when something seemingly unrelated had broken the DL path.

7.4 Unit Tests

The `dl-training-service` ships with 22 unit tests:

- Architecture forward-pass shapes — one test per architecture on a known input tensor.
- VRAM-guard envelope — parametrised over every documented `(arch, input_size, batch_size)` triple, asserts the estimate is under the 6 GB-minus-1 GB budget.
- Oversize refusal — a request with an obviously oversized batch is rejected by the route before training starts.
- End-to-end `build` → `save` → `predict` on a tiny LeNet against a 12-image fixture dataset, against a real torch instance (CPU on CI).
- Tier-ceiling enforcement — a free-tier token posting 20 epochs is rejected with the documented error code.

Conclusion

Sprint 4 was the riskiest sprint of the project. Bolting an entirely new workload onto a platform that had calmed down into three sprints of mostly-tabular and RAG work could have destabilised the whole thing. It didn't, mostly because the conventions established in earlier sprints — the shared-volume pattern, the Celery-per-service split, the gateway header injection — absorbed the new service without needing to bend.

Chapter 8

Sprint 5: Multi-tenancy, Project ACL, and Real-Time Collaboration

Introduction

Up to this point the platform had been a single-user experience. The company-tier story needed something else: multiple people on the same pipeline at the same time, with sensible access controls and enough real-time feedback that they wouldn't accidentally undo each other's work.

8.1 Sprint Backlog

- Per-pipeline access control with five distinct roles.
- Server-side and client-side role enforcement.
- Real-time canvas presence (cursors and node highlights).
- Threaded chat per pipeline.
- One-click Google Meet links (via Google OAuth).
- Per-pipeline notification fan-out (the bell in Sprint 6 will subscribe to this).

8.2 Design

PLACEHOLDER: Use case diagram for Sprint 5

Figure 8.1: Use case diagram — collaboration roles and actions

PLACEHOLDER: Sequence diagram — real-time presence

Figure 8.2: Sequence diagram — cursor + node-selection events over SocketIO

PLACEHOLDER: Class diagram for Sprint 5

Figure 8.3: Class diagram — pipeline membership, chat threads, Meet integration

8.2.1 The role hierarchy

- **Owner** — the user who created the pipeline. Cannot be removed.
- **Project Manager** — can invite, can edit, cannot remove the owner.
- **Data Scientist** — can edit nodes, can run training, cannot manage members.
- **Analyst** — can view metrics and download artefacts, cannot run training.
- **Viewer** — read-only.

The role is enforced both server-side and client-side. The gateway resolves the user's role for the project on every `/pipelines/*` request and stamps `X-Project-Role`; the frontend disables UI affordances based on the same role. The server-side enforcement is authoritative — the client-side disable is for UX, not security. Mixing those two up is one of the easier mistakes to make in this area.

PLACEHOLDER: Manage Access dialog

Figure 8.4: Manage Access dialog — invite, role change, remove

8.3 Implementation

8.3.1 Real-time presence

When a user opens a pipeline, the gateway joins them to the `pipeline_<id>` SocketIO room. Cursor positions and node selections are emitted as throttled (50 ms) events. The frontend maintains a per-user coloured cursor overlay on the canvas. There's a particular satisfaction in seeing another user's cursor move across your screen — it makes the platform feel inhabited in a way that chat alone never quite does.

PLACEHOLDER: Real-time presence screenshot

Figure 8.5: Real-time presence — two users editing the same pipeline

8.3.2 Pipeline chat and Google Meet

Each pipeline gets its own threaded chat, persisted to Postgres. An "@" mention fires a notification (which the Sprint 6 bell will have somewhere to surface). The Google Meet integration relies on Google's OAuth flow — not for any data-sharing reason, but because a Meet link generated server-side without OAuth wouldn't be tied to the inviting user's calendar, which made it useless for the actual workflow.

8.3.3 Why the gateway resolves the role

The role-resolution lookup happens inside the gateway rather than each downstream service. Two reasons:

1. The gateway already maintains the request-scoped user identity. Adding role resolution here keeps the zero-trust pattern intact — downstream services trust the stamped `X-Project-Role` the same way they trust `X-User-Id`.

2. It centralises the cache. A short-lived Redis-backed cache on `(user_id, pipeline_id)` → `role` cuts the auth-service lookup count to a small fraction of the request count, without spreading caching logic across four services.

8.4 Unit Tests

- Role-matrix table-test: for each `(role, action)` pair, the expected permit/deny outcome.
- Role demotion mid-session: a Project Manager has their role downgraded to Viewer while connected; the next canvas-edit event is rejected.
- Owner immutability: an attempt to remove the owner returns the documented error code.
- SocketIO presence: two test clients join the same pipeline room and assert that each sees the other's cursor events.

Conclusion

The collaboration layer turned the platform from a personal tool into something a team could share. The interesting part of this sprint was how little of the existing architecture had to change to support it — the gateway already injected user identity, the SocketIO bus was already in place for live training, and the role check fit naturally into the existing header convention.

Chapter 9

Sprint 6: Dashboard, i18n, Accessibility, and Admin Console

Introduction

Every project has the sprint where the punch list of small things finally gets crossed off. This was that sprint. The dashboard, the billing UI, internationalisation, the high-contrast theme, the notification bell, the admin operations dashboard, user impersonation, and undo/redo on the canvas — each modest on its own, but together they moved the platform from "demonstrably works" to "demonstrably operable".

9.1 Sprint Backlog

- User-facing dashboard (datasets, pipelines, model versions in one view).
- Billing UI (free / solo / company tier; Stripe checkout).
- Internationalisation (English + French, generated bindings).
- High-contrast theme + WCAG AA audit.
- Persistent notification bell.
- Admin operations dashboard (user management, audit log, queue + hardware monitors, migration drift panel).
- User impersonation with hard five-minute TTL.
- Undo / redo on the canvas.

9.2 Design

PLACEHOLDER: Use case diagram for Sprint 6

Figure 9.1: Use case diagram — dashboard + admin operations

PLACEHOLDER: Dashboard wireframe

Figure 9.2: Wireframe — user dashboard with datasets, pipelines, versions

PLACEHOLDER: Class diagram for Sprint 6

Figure 9.3: Class diagram — notifications, audit log, impersonation token

9.3 Implementation

9.3.1 The user dashboard and billing UI

PLACEHOLDER: Dashboard screenshot

Figure 9.4: User dashboard — datasets, pipelines, model versions, quota

The dashboard is a single page with three tabs (Datasets, Pipelines, Models), each backed by a paginated table with server-side sorting. The right rail shows the user's quota (datasets, model versions, RAG documents) for the current tier and a "Manage Plan" link to the billing UI. Billing itself goes through Stripe Checkout for purchases and the Stripe Customer Portal for upgrades and cancellations — the platform never handles a card number directly.

9.3.2 Internationalisation

Every user-facing string moved to `i18next`, with English and French resource bundles. The TypeScript bindings are generated from the English bundle, so a missing French key fails the type-check rather than rendering as a raw key string in production. That's a small piece of automation that paid for itself the first time I forgot to translate a label.

9.3.3 High-contrast theme and accessibility

A high-contrast theme was added for users with visual impairments, toggleable from the navbar. Every interactive element gained an `aria-label`; the colour palette was checked against WCAG AA contrast ratios. Accessibility isn't an optional extra at the company tier — in some jurisdictions it's a procurement requirement.

9.3.4 Notification bell with persistence

A bell icon in the navbar shows unread chat mentions, training-done events, training-failed events, document-indexed events, and meeting links. The store is persisted to `localStorage` so a page refresh doesn't drop the unread count, which sounds obvious until you build the first version and the count keeps resetting to zero.

9.3.5 Admin operations dashboard

The super-admin role gained a dedicated page with five panels:

PLACEHOLDER: Billing page screenshot

Figure 9.5: Billing page — tier selection, Stripe checkout

PLACEHOLDER: Language toggle in action

Figure 9.6: Language toggle — English / French

- **User management** — search, suspend, delete, override quotas.
- **Audit log viewer** — paginated table of every sensitive action (login, password reset, impersonation, suspension), with IP and JSON detail.
- **Queue monitor** — live stats from each Celery queue.
- **Hardware monitor** — CPU, RAM, GPU utilisation sampled from the metrics-service every five seconds.
- **Migration drift panel** — compares each service's latest Alembic revision against the database and surfaces the diff.

PLACEHOLDER: Admin dashboard screenshot

Figure 9.7: Admin operations dashboard — five-panel layout

9.3.6 User impersonation

Super-admins can "view as" any user from the user-management table. The auth-service mints a separate five-minute access token with an `imp_actor` claim identifying the original super-admin. A red banner persists across every page during the impersonation session, showing the target email and a live countdown to expiry. Clicking `Exit` (or hitting Logout in the navbar) restores the super-admin's original token. Two audit events are recorded: `admin.impersonate_start` and `admin.impersonate_end`.

The hard TTL is the load-bearing piece here. An impersonation session that doesn't time out is, sooner or later, an incident waiting to happen.

9.3.7 Undo / redo on the canvas

A 50-snapshot history stack with drag-coalescing (a single drag gesture produces one snapshot, not 60) and `Cmd/Ctrl+Z` plus `Cmd/Ctrl+Shift+Z` keyboard shortcuts. Two `IconButton`s sit next to the auto-layout button for users who don't know the keyboard shortcut. It's a small feature, but it's one of those things you don't notice until it's missing.

9.3.8 Cross-cutting changes

Two architectural shifts landed this sprint that don't fit neatly under any one feature:

PLACEHOLDER: Impersonation banner

Figure 9.8: Impersonation banner — target email + live countdown

- **Auth state moved to `sessionStorage`.** Before this sprint, refreshing the page during impersonation lost the original super-admin token — the Zustand auth store was in-memory only. Wrapping it in `persist(..., sessionStorage)` fixed the impersonation case and quietly improved the general refresh behaviour for normal users at the same time.
- **ConfirmDialog with typed confirmation.** Destructive admin actions (delete-user, delete-announcement) now require retyping the target's email or title before the action button enables. The native `confirm()` dialog was too easy to misclick during a live demo, and once was enough.

9.4 Unit Tests

- i18n key parity: a CI step diffs the English and French bundles and fails if a key is missing on either side.
- Notification reducer: state transitions for each event type (mention, training-done, etc.), plus persistence round-trip.
- Impersonation flow: the original token, the impersonation token, and the post-exit restore are asserted across three pytest cases.
- Audit-log presence: every sensitive route is exercised and the audit table is asserted non-empty for that action.
- Undo / redo: a sequence of canvas edits, then a fixed number of `Ctrl+Z` presses, then a deep-equality check against an earlier snapshot.

Conclusion

The punch-list sprint never makes for thrilling reading, but it's where the platform earned the right to be presented to a real user. The admin console in particular changed how I thought about the platform — having a single place to inspect queue health, hardware load, and migration state turned operations from a vague worry into a tractable one.

Chapter 10

Testing and Deployment

Introduction

The platform's testing and deployment stories evolved in parallel with the rest of the system, and by the end they looked nothing like what I had at the start. This chapter covers both — how the tests are organised, what they actually verify, and how the whole thing ships.

10.1 Test Strategy

10.1.1 Backend Testing

The test suite is split by service. Each Python service has a `tests/` directory with `pytest` fixtures that fake the backing services (Mongo via `mongomock`, Celery via a monkey-patched `apply_async`) so the suite runs in seconds on a CI runner without spinning up the real stack.

Coverage is unevenly distributed, which I won't pretend otherwise about:

- `auth-service` — the authentication path and the admin operations sit above 80% line coverage.
- `ml-training-service` — the algorithm-factory path is fully exercised; the live-streaming SocketIO path is not.
- `dl-training-service` — 22 unit tests covering the route's contract surface, the architecture forward-pass shapes, the VRAM-guard envelope, and an end-to-end `build` → `save` → `predict` happy path on a real torch instance.

PLACEHOLDER: Backend coverage report

Figure 10.1: Backend coverage report — per-service line coverage

10.1.2 Frontend Testing

The frontend uses `vitest` for unit tests but relies mostly on TypeScript strict mode plus ESLint as the structural guarantee. In practice more than 99% of regressions in this codebase are caught by `tsc -noEmit` before they ever reach a runner — typing is the cheapest test you can write.

PLACEHOLDER: Frontend coverage report

Figure 10.2: Frontend coverage report — vitest output

10.1.3 CI runs

PLACEHOLDER: GitHub Actions — backend job

Figure 10.3: GitHub Actions output for the backend lint + test job

PLACEHOLDER: GitHub Actions — frontend job

Figure 10.4: GitHub Actions output for the frontend lint + test job

End-to-end, CI runs in roughly six minutes for a frontend-only change, nine minutes for a backend change, and fourteen minutes when both sides move at once.

10.2 Deployment

The whole stack ships as a single `docker-compose.yml` with 14 services:

- 4 backing data stores (Postgres + pgvector, Mongo, Redis, TimescaleDB).
- 5 application services (gateway, auth, data, ml, dl).
- 3 Celery workers, one per CPU-bound service.
- 1 metrics collector.
- Ollama plus a one-shot `ollama-pull` init container.
- MailHog for development email capture.

A `make up` brings the entire platform online. First-run duration is dominated by the model pulls: Ollama fetches the LLM weights on first boot (around 2 GB), and the dl-training service pulls the CUDA wheels on first build (around 1 GB). After that, `make up` is a matter of seconds.

PLACEHOLDER: Deployment diagram

Figure 10.5: Deployment diagram — 14 containers, three named volumes

10.2.1 Volumes

Three named Docker volumes carry the binary artefacts that survive container restarts: `uploaded_files` (datasets and image zips), `model_artifacts` (trained models), and `ollama_models` (LLM weights). `uploaded_files` is mounted into the gateway, both ingestion containers (API and worker), the ml-training containers, and the dl-training containers — the shared-volume convention from Sprint 1 ended up applied very broadly, arguably more broadly than I'd originally planned.

10.2.2 Healthchecks and dependency ordering

Every backing service has a `healthcheck`, and every application service `depends_on` the relevant healthchecks with `condition: service_healthy`. The first time I deployed the stack without this discipline, the auth service started before Postgres had finished initialising and proceeded to crash-loop on the Alembic migration. After that, the healthcheck convention became non-negotiable for any service that touches a database.

10.2.3 Network and security topology

PLACEHOLDER: Network topology

Figure 10.6: Network topology — gateway as the single ingress, internal Docker network for the rest

The gateway is the only container exposed on the host. All other services bind to the internal Docker network. The Postgres instance is not reachable from outside the Compose stack; the only path to it is through the auth-service or the ml-training-service, and the only path to either of those is through the gateway. That is the same zero-trust posture as the JWT header injection, applied at the network layer.

10.2.4 Bugs encountered during deployment

A non-exhaustive list, in roughly the order they bit me:

- **Prophet missing CmdStan.** Prophet's probabilistic backend is a C++ binary that has to be compiled at image-build time, not pip-installed. The fix was a `python -c "import cmdstanpy; cmdstanpy.install_cmdstan()"` step in the ml-training Dockerfile.
- **Flask CLI app discovery inside containers.** Running `flask db migrate` via `docker compose exec` couldn't find the app factory; the fix was an explicit `-e FLASK_APP=app.main` on every CLI invocation, baked into the Makefile so I wouldn't forget.

- **React 18 Strict Mode and rate limiting.** Strict Mode double-invokes `useEffect` in development; the rate limit was tripping. Bumped the per-user limit to 600/min in development, kept the production cap tight.
- **torch CUDA wheels in CI.** The `dl-training-service` Dockerfile pulls CUDA-12.1 torch from PyTorch's index; on a CI runner that would have been a 1 GB download for no benefit. CI installs CPU-only torch wheels separately, gated on `matrix.service == 'dl-training-service'`.
- **Ollama GPU detection in Compose.** The default compose deployment runs Ollama on CPU. Enabling the `deploy.resources.reservations.devices` block requires `nvidia-container-toolkit` on the host, which isn't the default on Ubuntu Server. Documented in the README and commented out in the compose file by default.
- **DL VRAM-guard calibration.** The static per-pixel constants were tuned against documented Turing memory numbers rather than measured ones. The first real GPU run on my 1660 Super exceeded the estimate by about 15% on MobileNet at 224 px batch 32; the constants got nudged upwards accordingly. The unit-test suite tracks the envelope shape, not the absolute MB, so this calibration drift was caught empirically rather than by a failing test.

Conclusion

The stack is, by deployment-engineering standards, modest — one host, fourteen containers, `make up`. That simplicity is the point. A platform whose deployment story requires Kubernetes expertise wouldn't be a credible answer to the question this project set out to ask.

General Conclusion

The end-of-studies project set out to build a privacy-first, no-code AI platform that could stand alongside cloud-managed ML services without making the data-sovereignty compromises those services demand. After six sprints, the platform supports three pipeline modes — tabular ML with fourteen algorithms, fully-local RAG-augmented chat, and image-classification deep learning — runs on a single consumer machine with 6 GB of VRAM, and exposes that capability through a visual canvas designed for users who don't write Python.

Key deliverables

- **Five Python microservices** totalling roughly 6,500 lines of code across the api-gateway, auth-service, data-ingestion-service, ml-training-service, metrics-service, and dl-training-service. Three independent Celery worker processes handle the asynchronous workloads.
- **A React + TypeScript single-page application** of roughly 13,000 lines, including the React Flow canvas, the live training visualisations, the admin operations dashboard, the i18n bundles for English and French, the high-contrast theme, and the impersonation banner.
- **14 docker-compose services** (5 application services, 3 Celery workers, 4 backing data stores, Ollama, `ollama-pull`, MailHog).
- **14 tabular ML algorithms** covering classification, regression, clustering, and time-series forecasting.
- **A fully-local RAG pipeline** built on sentence-transformers, pgvector, and Ollama serving Llama 3.2 3B `Q4_K_M`.
- **3 deep-learning architectures** (LeNet, custom ResNet-9, MobileNet-V3-Small with optional ImageNet transfer learning), guarded by a static VRAM estimator that refuses oversize requests before the GPU sees them.
- **A GitHub Actions CI pipeline** with parallel lint and test jobs across all five Python services and the TypeScript frontend.
- **An Alembic migration history** of nine revisions across two services, idempotent and replayable.

Validation results

The platform was validated end-to-end on a representative tabular classification problem (a 250,000-row salary dataset, Random Forest regression, R^2 of 0.9785 in 4 min 12 s on the demo machine) and on a representative DL classification problem (the synthetic 30-image / 3-class dataset shipped as `seed_demo_image_dataset.py`, tiny_resnet at 64 px batch 32, 5 epochs, terminal validation accuracy of 100% in roughly 30 s on a 1660 Super).

PLACEHOLDER: Final results dashboard

Figure 10.7: Final results dashboard — tabular + DL validation runs

What worked, what I'd change

What worked. The single design decision that paid off most was the gateway-only header-injection rule from Sprint 1. Every downstream service can trust `X-User-Id` without inspection, which removed an entire category of cross-service auth bugs that a more permissive setup would have invited. The shared-volume convention was the second — it saved me from writing a streaming-upload protocol, and it generalised cleanly to RAG documents, image zips, and trained-model artefacts. The Celery-per-service split was the third — a long XGBoost run genuinely cannot block an authentication request, and a 20-epoch DL run cannot starve the fast tabular queue.

What I'd change. Flask over FastAPI: the absence of typed routes cost me hours over the project's life, especially in `ml-training-service` where the request bodies are large and nested. Three algorithm registries instead of one: the tabular registry, the architecture registry, and the algorithm-display catalogue all encode mostly the same information in slightly different shapes; a single config-driven catalogue would have been easier to maintain. A formal schema for socket events: the `/training` namespace grew organically and is now mildly inconsistent (some events use `step`, some use `epoch`, some use both). Lesson learned the slow way.

Limitations and future work

The platform is single-host by construction — the shared-volume convention assumes a single filesystem. Scaling out the workers would need either a network filesystem or an S3-compatible object store. Per-user quota overrides require a JWT refresh to take effect, because the tier rides in the JWT claim. The DL VRAM guard is calibrated against the demo machine; a truly portable estimator would instrument a one-time calibration run on first boot.

Future work: ONNX export for DL versions (the dl-training service already saves a clean state_dict — this is a one-file change away), object detection via a pre-built YOLOv8 model for inference only, per-user quota override invalidation via a Redis-backed override cache the gateway consults, and a typed schema for the `LiveTrainingChart` socket events.

Closing remark

What I take away from this project is a specific belief: the common framing that "cloud AI is the only way you can scale ML" is not exactly wrong, but it understates how much can be done locally. A laptop with 6 GB of VRAM, a competent CPU, and 16 GB of RAM is enough to host an AutoML platform, a small generative-AI workspace, and a deep-learning training loop — all at once, and without any of the data ever leaving the machine. For the regulated industries this project was framed against, that's not a marginal improvement. It's the difference between machine learning being usable at all and not.

Bibliography

[1] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[2] T. Chen and C. Guestrin, *XGBoost: A Scalable Tree Boosting System*, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016.

[3] S. M. Lundberg and S.-I. Lee, *A Unified Approach to Interpreting Model Predictions*, Advances in Neural Information Processing Systems 30 (NeurIPS 2017).

[4] S. J. Taylor and B. Letham, *Forecasting at Scale*, The American Statistician, 2018.

[5] W. Wang et al., *MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers*, NeurIPS 2020.

[6] P. Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020.

[7] Meta AI, *Llama 3.2: Open foundation and instruction models*, `https://ai.meta.com/llama/`, 2024.

[8] A. Howard et al., *Searching for MobileNetV3*, Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2019.

[9] K. He, X. Zhang, S. Ren, and J. Sun, *Deep Residual Learning for Image Recognition*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[10] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

[11] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS 2019.

[12] Ollama, *Get up and running with large language models locally*, `https://ollama.com/`, 2024.

[13] A. Kane, *pgvector: Open-source vector similarity search for Postgres*, `https://github.com/pgvector/pgvector`, 2024.

[14] xyflow GmbH, *React Flow Documentation*, `https://reactflow.dev/`, 2024.

[15] Celery Project, *Celery: Distributed Task Queue*, `https://docs.celeryq.dev/`, 2024.